Fatima Jinnah Women University

Department of Computer Sciences

Lab 06: Set Up Nexus Repository and Configure User Access

Course	DevOps (CS-649 Cloud DevOps)
Instructor	Sir Waqas Saleem
Student Name	Kiran Siddique
Registration No.	2023-BCS-052
Section	A
GitHub Username	KiranSiddique
Submission Repository	DevOps (Labs/Lab06)
Date of Submission	11 November 2026

Lab Overview

Item	Detail
Lab	Lab 06: Set Up Nexus Repository and Configure User Access
Submission directory	DevOps / Labs/Lab06
Server	Ubuntu Server 26.04.1 LTS, user kiransiddique, hostname ubuntu, IP 192.168.232.131
Docker Engine	29.1.3
Docker Compose	v5.7.0
Git	2.53.0
Nexus Repository	sonatype/nexus3:latest, Community Edition 3.98.1-01
Nexus ports	8006 (web and REST API), 8007 (Docker connector)
Maven	3.9.12 (Java 21.0.12.1 as system JDK)
Gradle	7.0 (Gradle Wrapper) running on JDK 16.0.2

Task List

Task 1: Run Nexus Repository and Complete the Initial Setup

Task 2: Create a Role and User and Verify Access

Task 3: Build and Publish the Gradle Application to Nexus

Task 4: Publish the Maven Application to Nexus

Task 5: Inspect Nexus Repositories and Components Using the REST API

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

Task 7: Create and Run a Maven Snapshot Cleanup Policy

Summary

Task 1: Run Nexus Repository and Complete the Initial Setup

I started Nexus Repository in a Docker container with a named volume for persistent data. Host port 8006 maps to the Nexus web interface and host port 8007 is reserved for the Docker connector used in Task 6. The image pull was slow and repeatedly reset by the network, so I forced IPv4 for the registry check and restarted the pull. Once the image was available, the existing container was started with docker start. I completed the setup wizard, replaced the generated administrator password, and disabled anonymous access. The one-time password was read privately and is not recorded in this report.

```
docker run -d --name nexus --restart unless-stopped \
  -p 8006:8081 -p 8007:8007 -v nexus-data:/nexus-data \
  sonatype/nexus3:latest
docker start nexus
docker ps | grep nexus
docker logs -f nexus
curl -i http://192.168.232.131:8006/service/rest/v1/repositories
```

The last request returned HTTP/1.1 401 Unauthorized, which confirms that anonymous access is disabled.

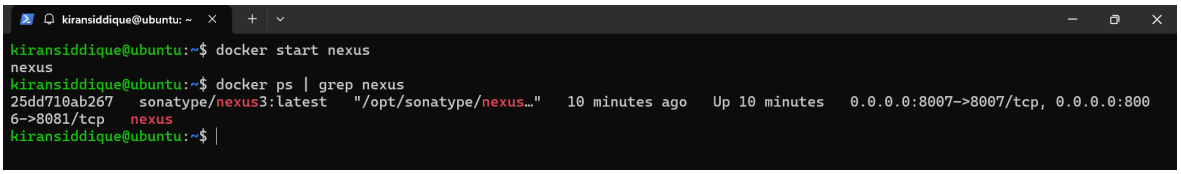


Figure 1: Running nexus container with ports 8006 and 8007 published (nexus_container_running.png)

Task 2: Create a Role and User and Verify Access

I created the nx-java role with the privilege nx-repository-view-maven2-maven-snapshots-*. The role grants read and write access to the Maven snapshots view only. I then created the active local user nexus with the nx-java role granted. I verified the account through the Nexus Status API. The password was entered at the curl prompt and is not shown.

```
curl -i -u nexus http://192.168.232.131:8006/service/rest/v1/status
Enter host password for user 'nexus':
HTTP/1.1 200 OK
```

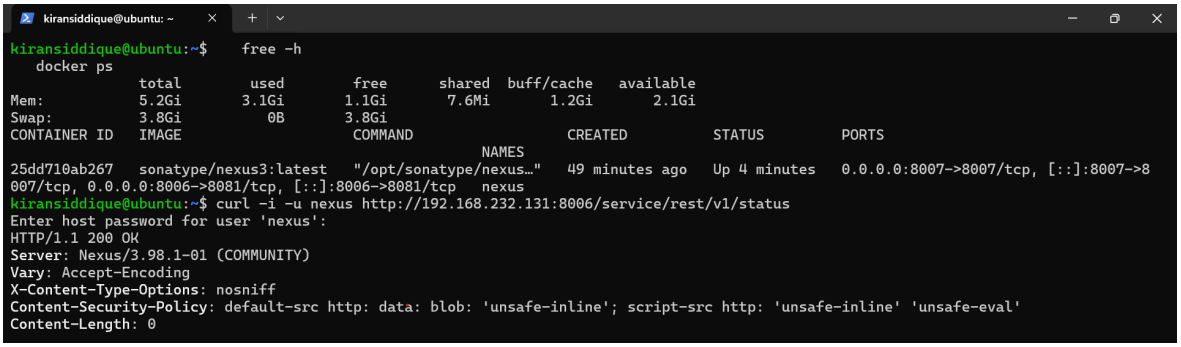


Figure 2: Authenticated Status API request returning HTTP/1.1 200 OK (nexus_user_access.png)

Task 3: Build and Publish the Gradle Application to Nexus

The java-app Gradle project from Lab 5 was used at /config/IdeaProjects/java-app. Its JDK was set to 16.0.2, which the Gradle 7.0 wrapper requires. I added the maven-publish plugin and a publication that targets the maven-snapshots repository over HTTP. I set the project name to my-app so the generated JAR is named my-app-1.0-SNAPSHOT.jar. The deployment credentials are stored in a project-level gradle.properties file with mode 600. Its content is not recorded here.

```
cd /config/IdeaProjects/java-app
./gradlew --version          # JVM: 16.0.2
./gradlew clean build        # BUILD SUCCESSFUL
./gradlew clean build publish  # BUILD SUCCESSFUL
```

settings.gradle contains rootProject.name = 'my-app'. The publishing block in *build.gradle* points to http://192.168.232.131:8006/repository/maven-snapshots/ and reads the repoUser and repoPassword properties.

```
kiransiddique@ubuntu:~/IdeaProjects/java-app$ cat settings.gradle
rootProject.name = 'my-app'
kiransiddique@ubuntu:~/IdeaProjects/java-app$ cat build.gradle
plugins {
    id 'java'
    id 'maven-publish'
    id 'org.springframework.boot' version '2.2.2.RELEASE'
    id 'io.spring.dependency-management' version '1.0.8.RELEASE'
}

group 'com.example'
version '1.0-SNAPSHOT'

publishing {
    publications {
        maven(MavenPublication) {
            artifact("build/libs/my-app-$version" + ".jar") {
                extension "jar"
            }
        }
    }
    repositories {
        maven {
            name "nexus"
            url "http://192.168.232.131:8006/repository/maven-snapshots/"
            allowInsecureProtocol = true
            credentials {
                username project.repoUser
                password project.repoPassword
            }
        }
    }
}

sourceCompatibility = 1.8

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation group: 'net.logstash.logback', name: 'logstash-logback-encoder', version: '5.2'
    testImplementation group: 'junit', name: 'junit', version: '4.12'
    implementation("org.elasticsearch:elasticsearch:7.0.0")
}
```

Figure 3: settings.gradle and the updated build.gradle with the Nexus publication (gradle_nexus_configuration.png)

```
kiransiddique@ubuntu:~/IdeaProjects/java-app$ ./gradlew clean build
Starting a Gradle Daemon, 1 incompatible Daemon could not be reused, use --status for details

BUILD SUCCESSFUL in 52s
4 actionable tasks: 4 executed
<-------------> 0% WAITING
> IDLE
kiransiddique@ubuntu:~/IdeaProjects/java-app$ ls build/libs/
my-app-1.0-SNAPSHOT.jar
kiransiddique@ubuntu:~/IdeaProjects/java-app$
```

Figure 4: ./gradlew clean build finishing with BUILD SUCCESSFUL and my-app-1.0-SNAPSHOT.jar in build/libs (gradle_nexus_build.png)

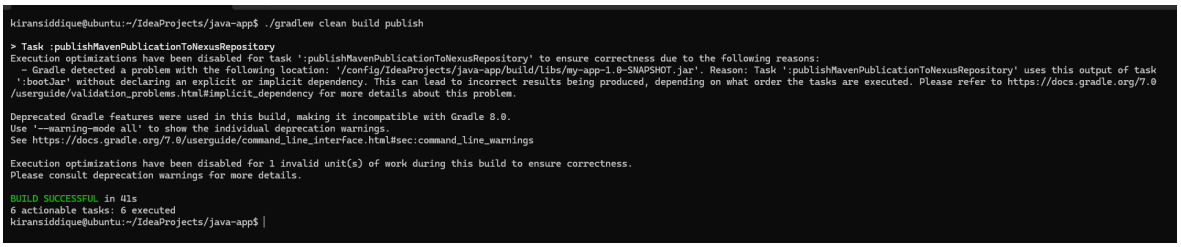


Figure 5: ./gradlew clean build publish finishing with BUILD SUCCESSFUL (gradle_nexus_publish.png)

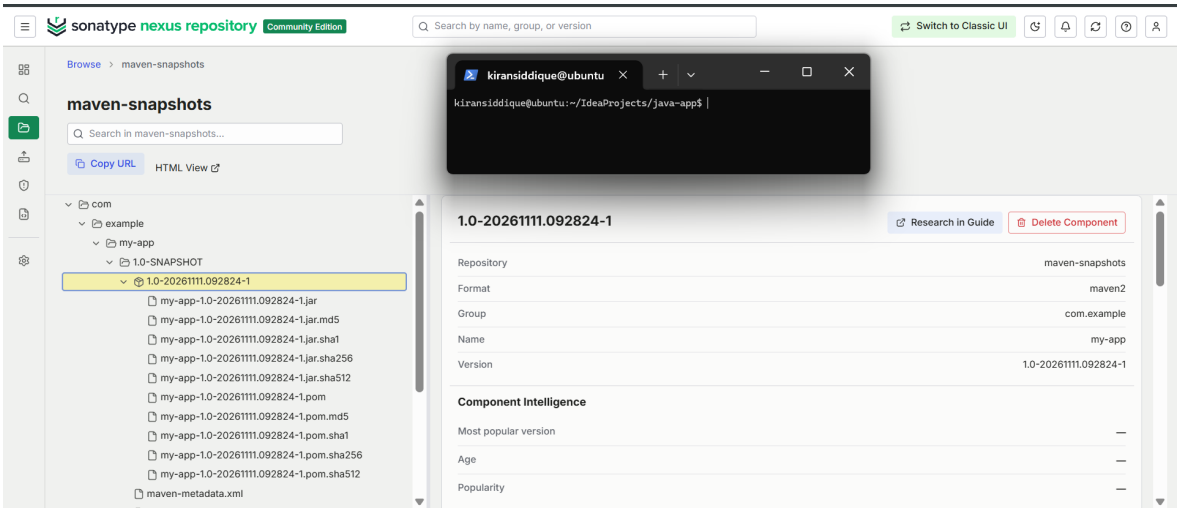


Figure 6: maven-snapshots repository showing the my-app artifact under com/example/my-app/1.0-SNAPSHOT (nexus_gradle_artifact.png)

Task 4: Publish the Maven Application to Nexus

I opened the java-maven-app project from Lab 5 and added a distributionManagement block to pom.xml. The block uses the snapshot repository ID nexus-snapshots, which must match the server entry in the Maven settings file. The settings file stores the nexus user's password with mode 600. The password is not recorded here. I then packaged the application and deployed the snapshot JAR, POM and metadata to Nexus.

```
cd /config/IdeaProjects/java-maven-app
sed -n '1,25p' pom.xml
mvn package          # BUILD SUCCESS, creates target/java-maven-app-1.1.0-SNAPSHOT.jar
mvn deploy           # BUILD SUCCESS, uploads to nexus-snapshots
```

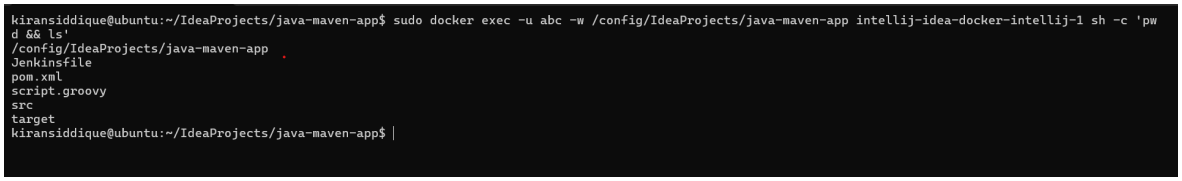


Figure 7: Working directory of the java-maven-app project with its files listed (maven_nexus_project.png)

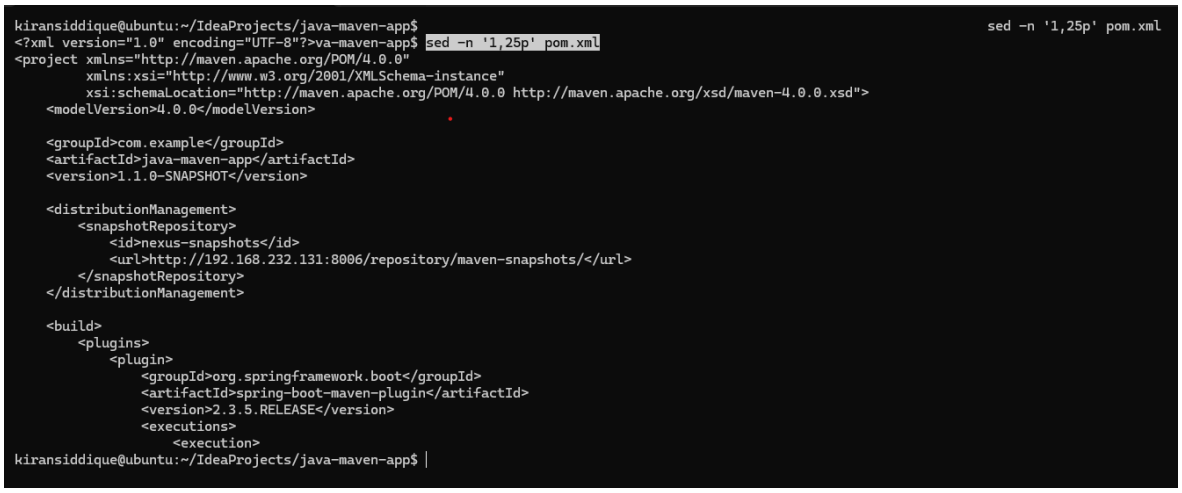


Figure 8: pom.xml with the distributionManagement block placed after the project version (maven_distribution_management.png)

```
kiransiddique@ubuntu:~/IdeaProjects/java-maven-app$ mvn package
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.example:java-maven-app >----------------------
[INFO] Building java-maven-app 1.1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ java-maven-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.6.0:compile (default-compile) @ java-maven-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ java-maven-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /config/IdeaProjects/java-maven-app/src/test/resources
[INFO]
[INFO] --- compiler:3.6.0:testCompile (default-testCompile) @ java-maven-app ---
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:2.17:test (default-test) @ java-maven-app ---
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.1.2:jar (default-jar) @ java-maven-app ---
[INFO] Building jar: /config/IdeaProjects/java-maven-app/target/java-maven-app-1.1.0-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:2.3.5.RELEASE:repackage (default) @ java-maven-app ---
[INFO] Replacing main artifact with repackaged archive
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  13.202 s
[INFO] Finished at: 2026-11-11T14:41:49+05:00
[INFO] ------------------------------------------------------------------------
kiransiddique@ubuntu:~/IdeaProjects/java-maven-app$
```

Figure 9: mvn package finishing with BUILD SUCCESS (maven_package_nexus.png)

```
kiransiddique@ubuntu:~/IdeaProjects/java-maven-app$ mvn deploy
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.example:java-maven-app >----------------------
[INFO] Building java-maven-app 1.1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.pom (8.5 kB at 3.6 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/42/maven-plugins-42.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/42/maven-plugins-42.pom (7.7 kB at 12 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/42/maven-parent-42.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/42/maven-parent-42.pom (50 kB at 130 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/32/apache-32.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/32/apache-32.pom (24 kB at 88 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.10.2/junit-bom-5.10.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.10.2/junit-bom-5.10.2.pom (5.6 kB at 4.5 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-install-plugin/3.1.2/maven-install-plugin-3.1.2.jar (32 kB at 58 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.pom (9.6 kB at 16 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-deploy-plugin/3.1.2/maven-deploy-plugin-3.1.2.jar (40 kB at 36 kB/s)
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ java-maven-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.6.0:compile (default-compile) @ java-maven-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ java-maven-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /config/IdeaProjects/java-maven-app/src/test/resources
[INFO]
[INFO] --- compiler:3.6.0:testCompile (default-testCompile) @ java-maven-app ---
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:2.17:test (default-test) @ java-maven-app ---
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.1.2:jar (default-jar) @ java-maven-app ---
[INFO]
[INFO] --- spring-boot:2.3.5.RELEASE:repackage (default) @ java-maven-app ---
[INFO] Replacing main artifact with repackaged archive
[INFO]
[INFO] --- install:3.1.2:install (default-install) @ java-maven-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.18/maven-resolver-util-1.9.18.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.18/maven-resolver-util-1.9.18.pom (2.9 kB at 7.9 kB/s)
```

Figure 10: mvn deploy uploading the snapshot JAR, POM and metadata, finishing with BUILD SUCCESS (maven_deploy_nexus.png)

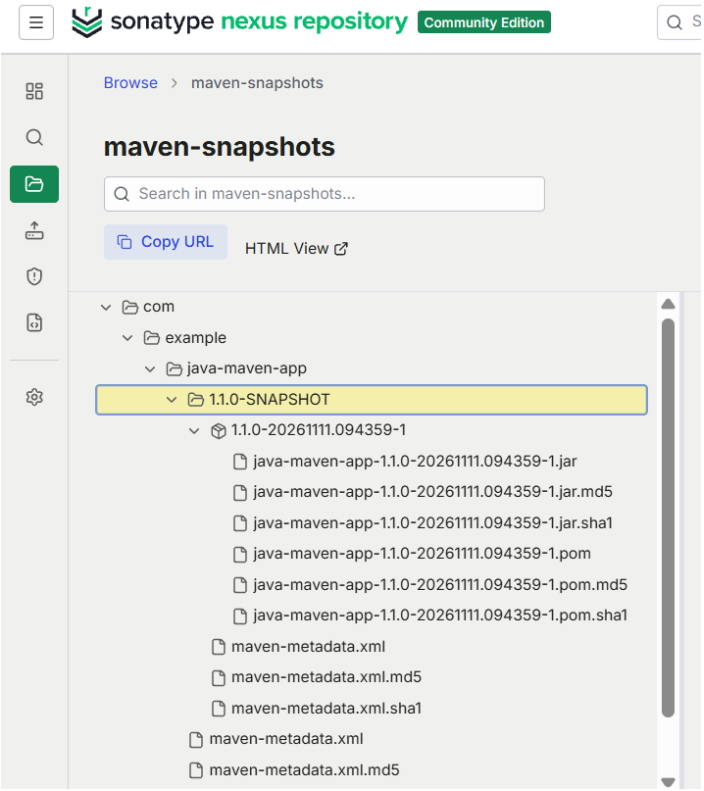


Figure 11: maven-snapshots repository showing the java-maven-app artifact under com/example/java-maven-app/1.1.0-SNAPSHOT (nexus_maven_artifact.png)

Task 5: Inspect Nexus Repositories and Components Using the REST API

I queried the Nexus REST API from the Ubuntu server terminal. Each request used curl with -u and a username only, so curl prompted for the password securely. I compared the repository list returned to the restricted nexus user with the list returned to the admin user. I then listed the components in the maven-snapshots repository and retrieved one component by its ID.

```
curl -u nexus -X GET "http://192.168.232.131:8006/service/rest/v1/repositories"
curl -u admin -X GET "http://192.168.232.131:8006/service/rest/v1/repositories"
curl -u nexus -X GET "http://192.168.232.131:8006/service/rest/v1/components?repository=maven-snapshots"
curl -u nexus -X GET "http://192.168.232.131:8006/service/rest/v1/components/<component-id>"
```

```
kiransiddique@ubuntu:~/IdeaProjects/java-maven-app$ curl -u nexus -X GET "http://192.168.232.131:8006/service/rest/v1/reposi
tories"
Enter host password for user 'nexus':
[ {
  "name" : "maven-snapshots",
  "format" : "maven2",
  "type" : "hosted",
  "url" : "http://192.168.232.131:8006/repository/maven-snapshots",
  "online" : true,
  "size" : 0,
  "attributes" : { }
} ]kiransiddique@ubuntu:~/IdeaProjects/java-maven-app$
```

Figure 12: Repository list returned to the nexus user (nexus_repositories_user.png)

```
kiransiddique@ubuntu:~/IdeaProjects/java-maven-app$ curl -u admin -X GET "http://192.168.232.131:8006/service/rest/v1/repositories"
Enter host password for user 'admin':
[ {
  "name" : "maven-releases",
  "format" : "maven2",
  "type" : "hosted",
  "url" : "http://192.168.232.131:8006/repository/maven-releases",
  "online" : true,
  "size" : 0,
  "attributes" : { }
}, {
  "name" : "maven-snapshots",
  "format" : "maven2",
  "type" : "hosted",
  "url" : "http://192.168.232.131:8006/repository/maven-snapshots",
  "online" : true,
  "size" : 0,
  "attributes" : { }
}, {
  "name" : "maven-central",
  "format" : "maven2",
  "type" : "proxy",
  "url" : "http://192.168.232.131:8006/repository/maven-central",
  "online" : true,
  "size" : 0,
  "attributes" : {
    "proxy" : {
      "remoteUrl" : "https://repo1.maven.org/maven2/"
    }
  }
}, {
  "name" : "maven-public",
  "format" : "maven2",
  "type" : "group",
  "url" : "http://192.168.232.131:8006/repository/maven-public",
  "online" : true,
```

Figure 13: Repository list returned to the admin user (nexus_repositories_admin.png)

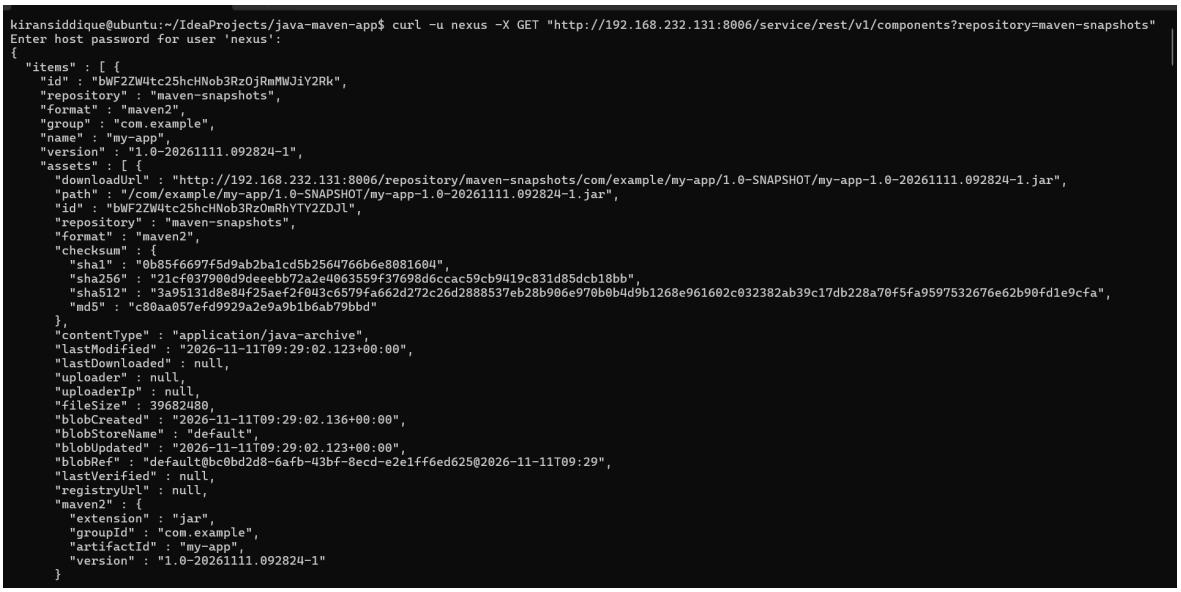


Figure 14: Components in the maven-snapshots repository, showing a component ID (nexus_components_list.png)



Figure 15: Details of one component retrieved by its ID (nexus_component_details.png)

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

I opened a shell in the Nexus container and read the default blob store directory without changing any file. I then created the file blob store mystore at /nexus-data/blobs/mystore. I confirmed its directory from the same container shell. Finally I created the docker-hosted repository, which is online, uses the mystore blob store and listens on HTTP connector 8007.

```
sudo docker exec -it nexus /bin/sh
pwd
cd sonatype-work/nexus3/blobs
ls
ls default/content
ls mystore
exit
```

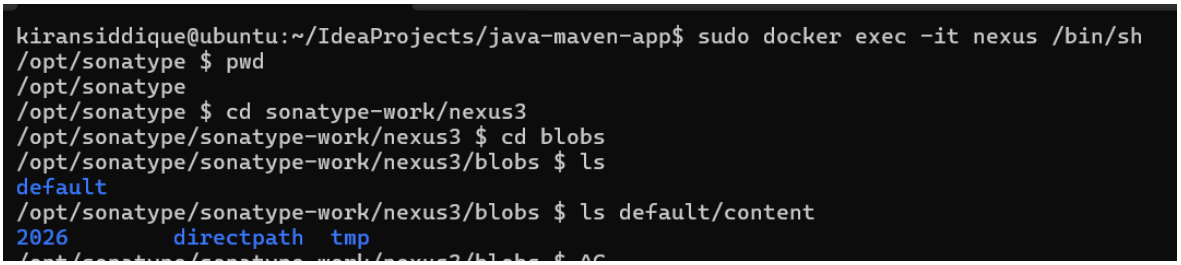


Figure 16: Blob store directory structure inside the Nexus container, with the default store and its content (nexus_default_blob_content.png)

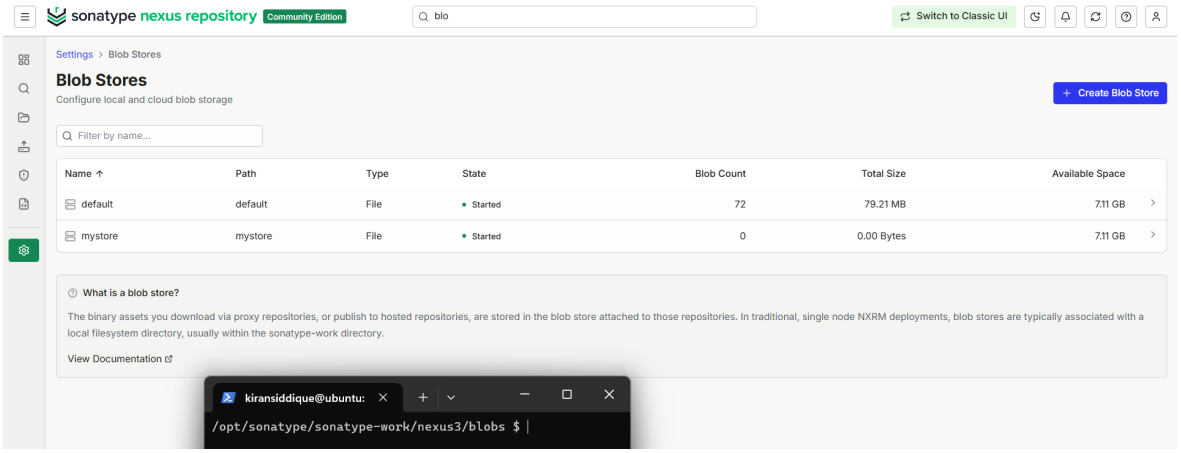


Figure 17: Blob Stores page showing default and mystore, both File type and Started (nexus_mystore_created.png)

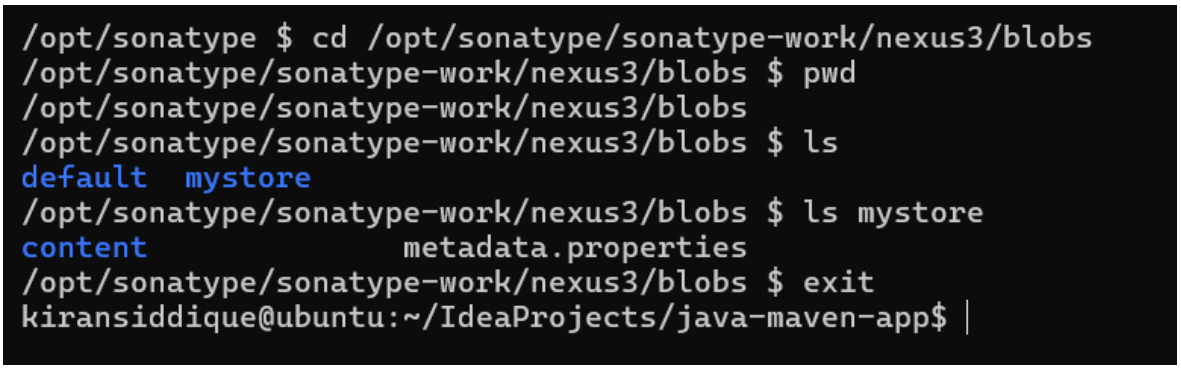


Figure 18: Container shell showing the blobs path, default and mystore directories, and the prompt after exit (nexus_blob_directories.png)

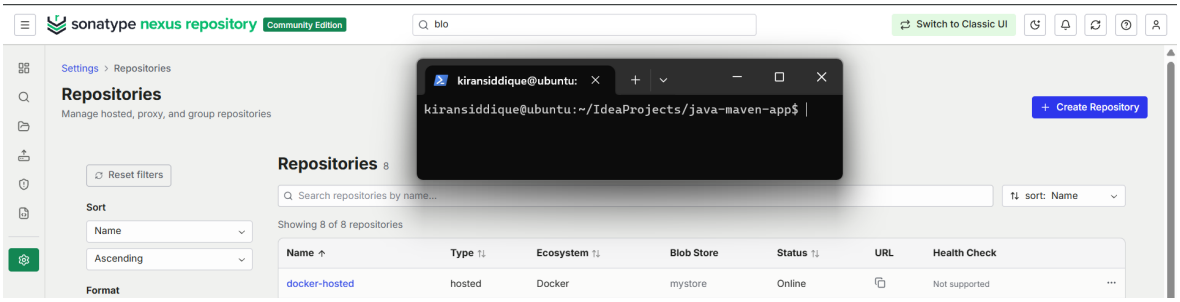


Figure 19: docker-hosted repository, Online, using the mystore blob store (nexus_docker_hosted_repository.png)

Task 7: Create and Run a Maven Snapshot Cleanup Policy

I created the maven cleanup policy for the maven2 format with the Pre-Releases/Snapshots release type and the .* asset-name matcher, then previewed it against maven-snapshots. I attached the policy to maven-snapshots. The cleanup service was scheduled for 6:00 AM and then run manually. Afterward, maven-snapshots showed no items. This removed the soft-deleted components from the browse view, so I recorded the default blob store before compaction. I then created the compact-store task for the default blob store with 0 days and a daily 06:30 schedule, ran it manually, and compared the default blob store values afterward.

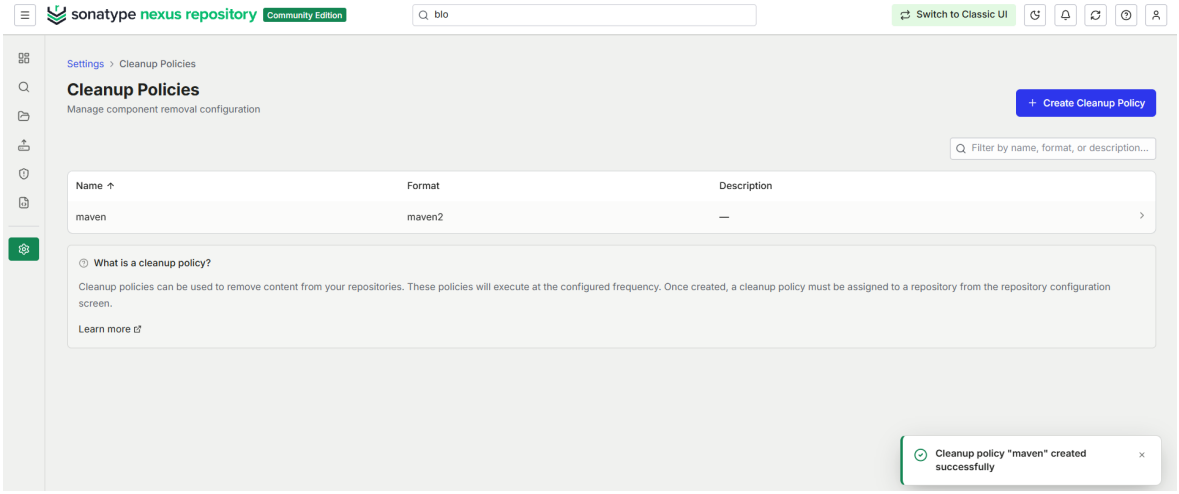


Figure 20: maven cleanup policy saved with the maven2 format and snapshot release type (nexus_cleanup_policy_created.png)

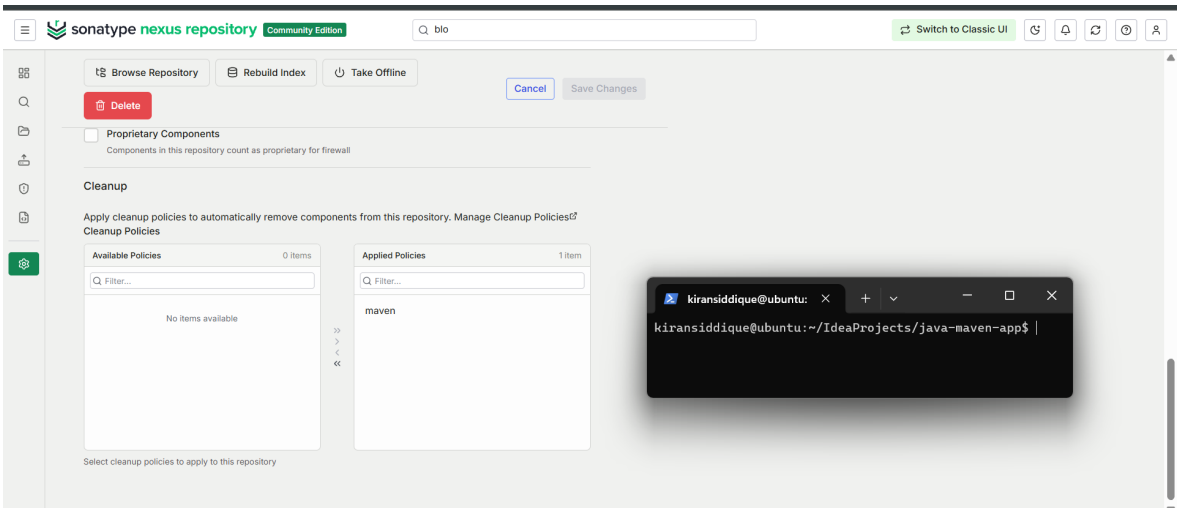


Figure 21: maven-snapshots repository with the maven policy in the Applied list (nexus_cleanup_policy_attached.png)

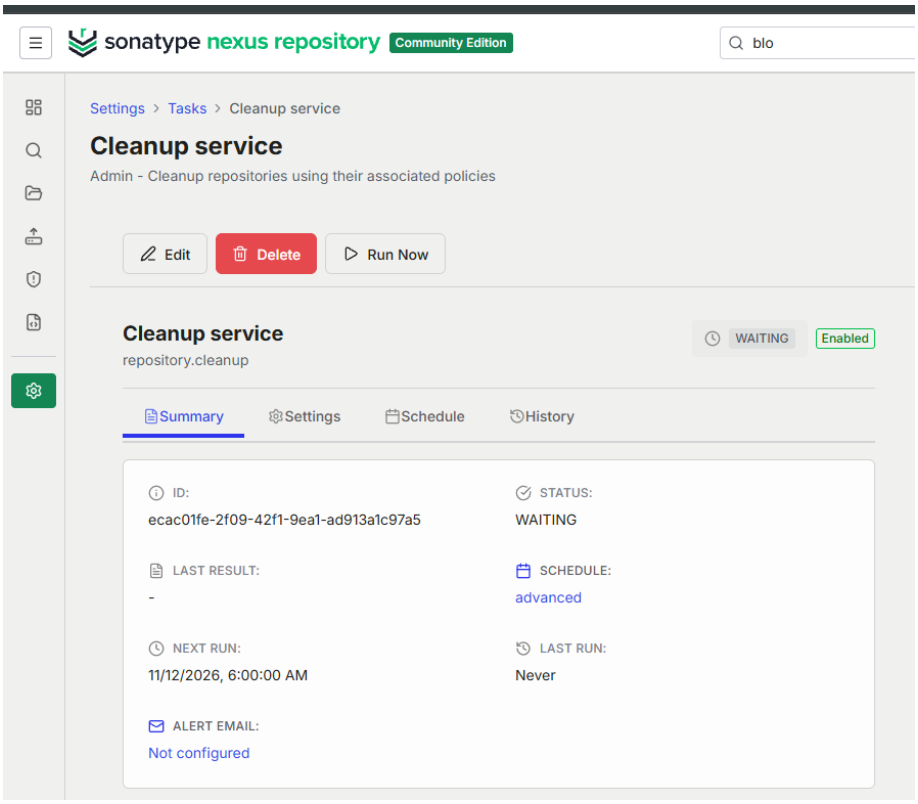


Figure 22: Cleanup service in Waiting state with the next run scheduled for 6:00 AM (nexus_cleanup_task_waiting.png)

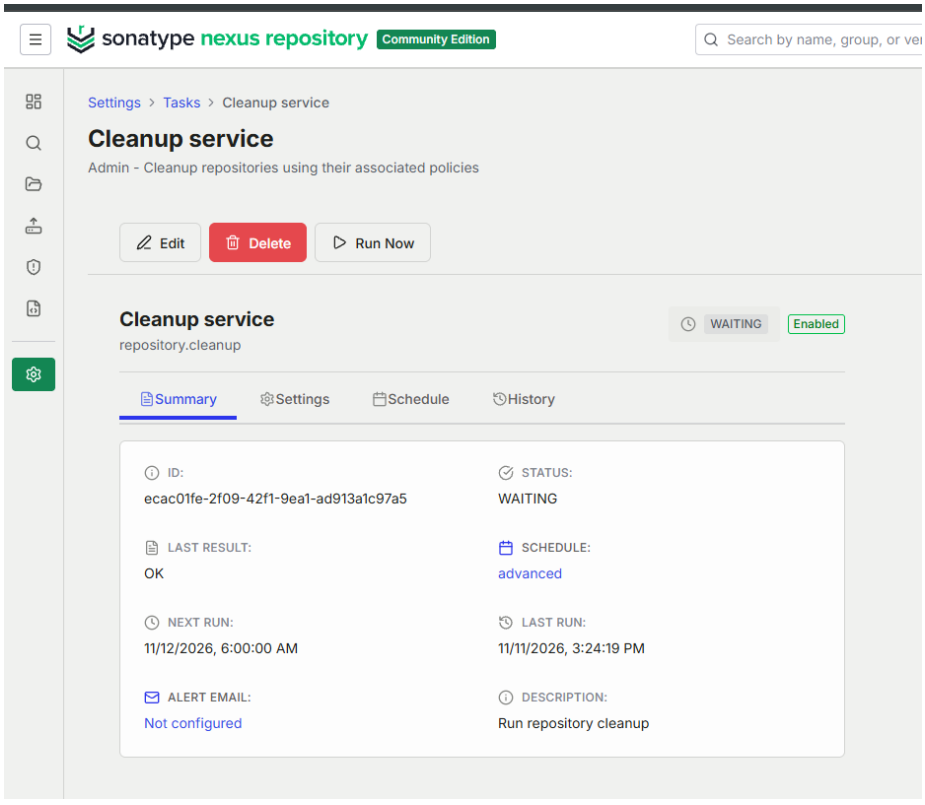


Figure 23: Cleanup service after the manual run, with an updated Last run and Last result OK (nexus_cleanup_task_completed.png)

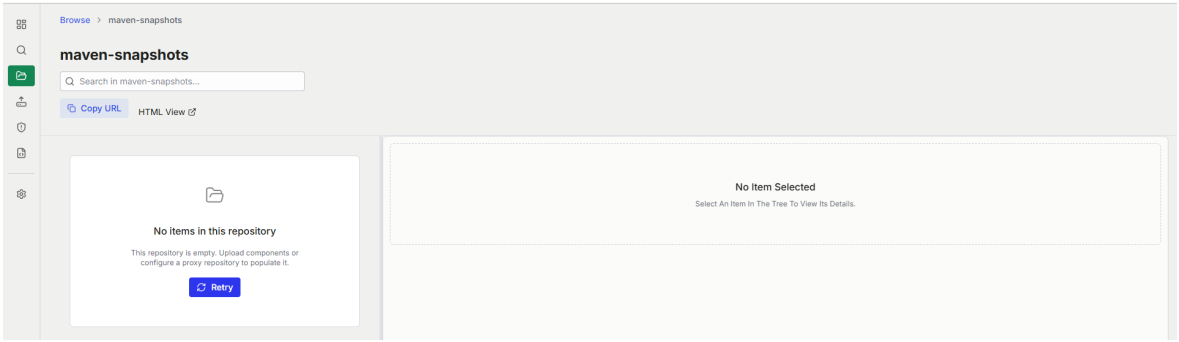


Figure 24: maven-snapshots repository with no items after the cleanup run (nexus_maven_snapshots_empty.png)

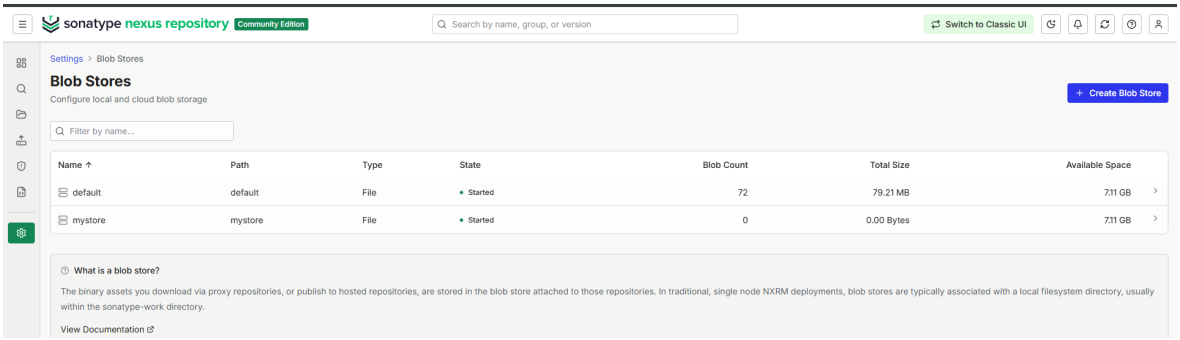


Figure 25: Default blob store values before compaction (nexus_blob_store_before_compaction.png)

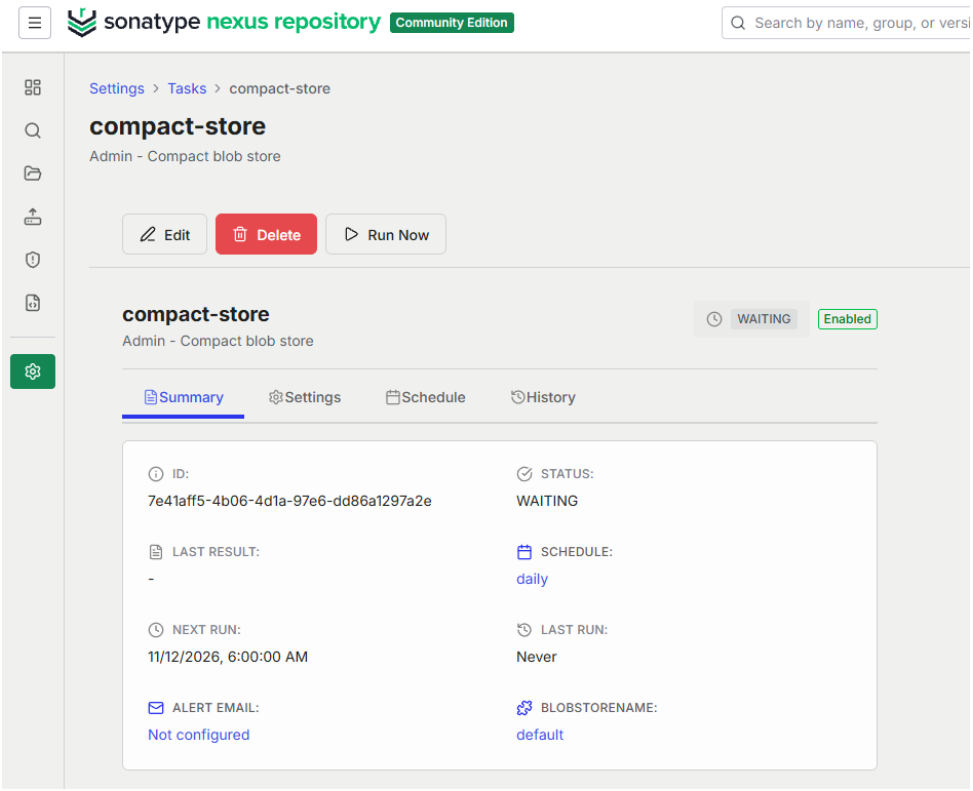


Figure 26: compact-store task saved for the default blob store, 0 days, daily at 06:30 (nexus_compact_task_configured.png)

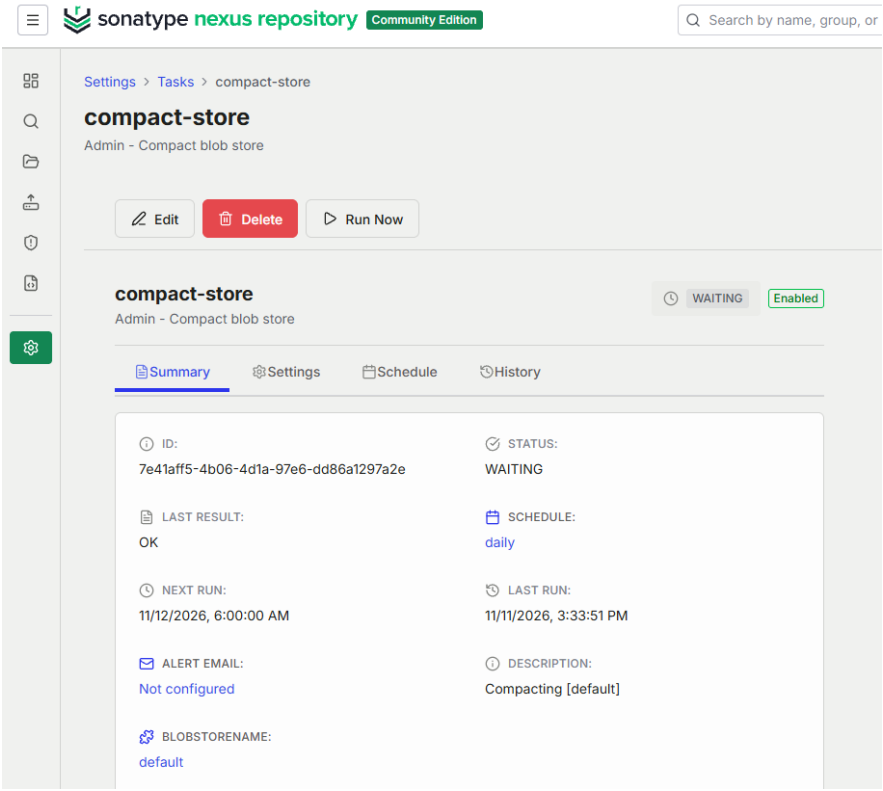


Figure 27: compact-store task after the manual run, with Last result OK (nexus_compact_task_completed.png)

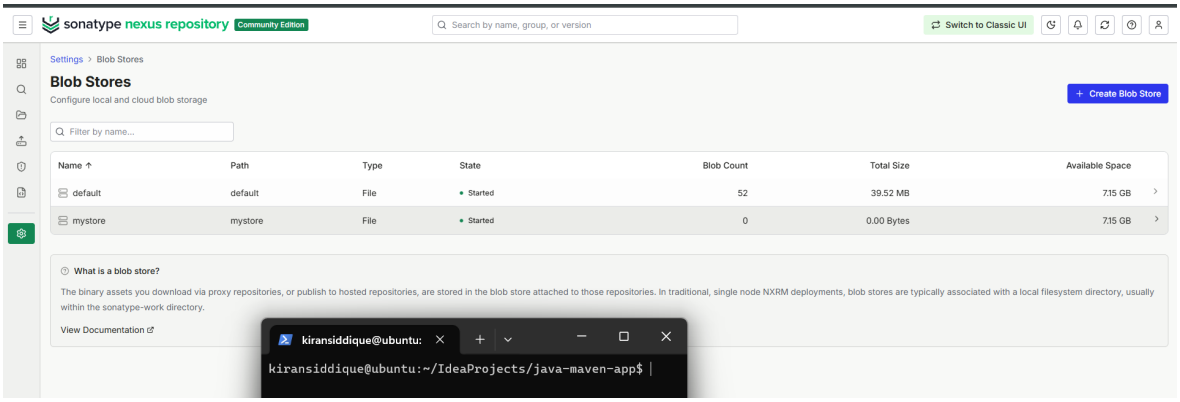


Figure 28: Default blob store values after compaction, with reduced used size and more available space (nexus_blob_store_after_compaction.png)

Summary

In this lab I ran Nexus Repository in Docker with persistent storage and completed its setup. I disabled anonymous access, created the nx-java role with Maven snapshot privileges, and created the active nexus user. I verified that user through the Status API. I published the Gradle my-app snapshot and the Maven java-maven-app snapshot to maven-snapshots, and checked both through the browser and the REST API. I created the mystore blob store and the docker-hosted repository that uses it. Finally I attached a snapshot cleanup policy, ran the cleanup service, and compacted the default blob store to reclaim disk space. Passwords and credential files are not recorded in this report.